Register Transfer and Microoperations

Unit 1-B

CHAITANYA PALLI

C R Rao AIMSCS

Register Transfer Language (RTL)

Register

- A processor register (CPU register) is one of a small set of data holding places that are part of the computer processor.
- A register may hold an instruction, a storage address, or any kind of data (such as a bit sequence or individual characters).
- Some instructions specify registers as part of the instruction. For example, an instruction may specify that the contents of two defined registers be added together and then placed in a specified register.

Register Transfer Language (RTL)

Register

- A register must be large enough to hold an instruction - for example, in a 64-bit computer, a register must be 64 bits in length.
- In some computer designs, there are smaller registers

  -for example, *half-registers* - for shorter instructions.
- Depending on the processor design and language rules, registers may be numbered or have arbitrary names.
- Register is the storage device, inside CPU, of data on which microoperations are performed.

Register Transfer Language (RTL)

Register

- The operations executed on data stored in registers are called microoperations.
- A microoperation is an elementary operation performed on the information stored in one or more registers.
- The result of the operation may replace the previous binary information of a register or may be transferred to another register.

Examples of microoperations are **shift, count, clear and load**.

- The internal hardware organization of a digital computer is best defined by specifying:

- The set of registers it contains and their function.
- The sequence of microoperations performed on the binary information stored in the registers.
- The control that initiates the sequence of microoperations.

Register Transfer Language (RTL)

Register

- The language, which is basically used to express the transfer of data among the registers, is called ***Register Transfer Language (RTL).***
- ***It is the symbolic notation used to describe the microoperation transfers among registers.***
- ***In such transfer, one of the source or destination should be register (not necessarily both).***

Register Transfer Language (RTL)

- Register Transfer Language, RTL, (sometimes called register transfer notation) is a powerful high level method of describing the architecture of a circuit.
- VHDL code and schematics are often created from RTL.
- RTL describes the transfer of data from register to register, known as microinstructions or microoperations.
- Transfers may be conditional.
- Computer registers are designated by **capital letters**.

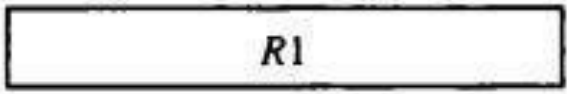
Register Transfer

- For example, the register that holds an address for the memory unit is usually called a **memory address register – MAR.**
- Other designations for registers are PC (for program counter), IR (for instruction register, and R1 (for processor register).
- The most common way to represent a register is by a rectangular box with the name of the register inside. Fig (a).
- The individual flip-flops in an n-bit register are numbered in sequence from 0 through n-1, starting from 0 in the rightmost position and increasing the numbers toward the left. For e.g. 8-bit register numbered: Fig (b).

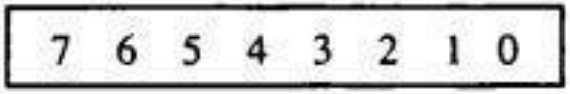
Register Transfer

- The numbering of bits in a 8-bit register can be marked on top of the box. Fig (c).
- A 16-bit register is partitioned into two parts in (d). Bits 0 through 7 are assigned the symbol L (for low byte) and bits 8 through 15 are assigned the symbol H (for high byte).
- The name of the 16 bit register is PC.
- The symbol PC(0-7) or PC(L) refers to the low order byte and PC(8-15) or PC(H) to the high order byte.

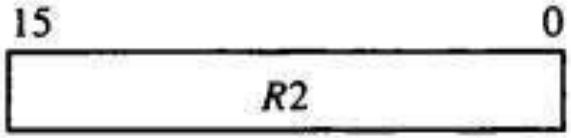
Register Transfer



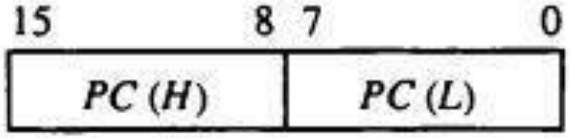
(a) Register R



(b) Showing individual bits



(c) Numbering of bits



(d) Divided into two parts

Register Transfer

- Information transfer from one register to another is designated in symbolic form by means of a replacement operator.

$$R2 \leftarrow R1$$

- The statement denotes a transfer of the content of register R1 into register R2. It designates a replacement of the content of R2 by the content of R1. By definition, the content of the source register R1 does not change after the transfer.

Register Transfer

- Information transfer from one register to another is designated in symbolic form by means of a replacement operator.

$$R2 \leftarrow R1$$

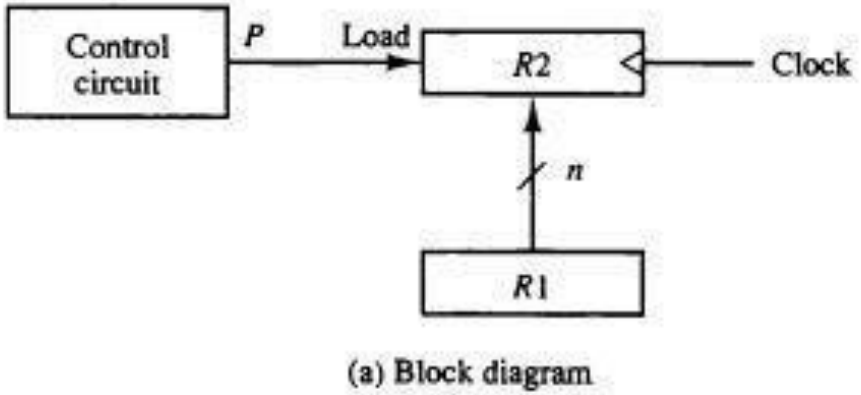
- If there is predetermined control condition like

If (P=1) then (R2 ← R1) then we can write the statement as

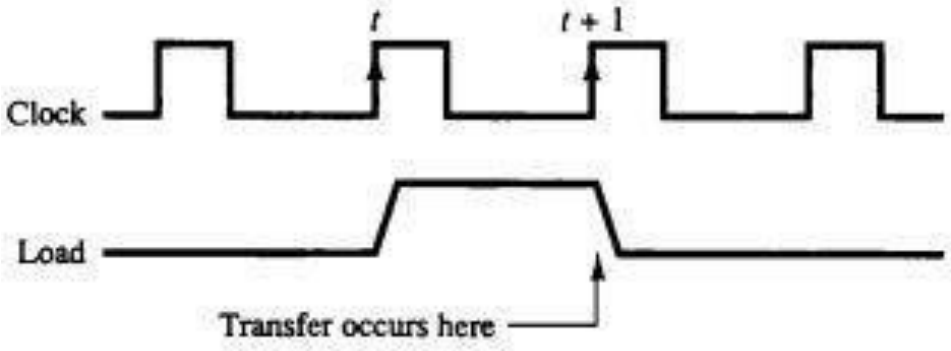
$$P: R2 \leftarrow R1$$

where P is control signal usually a control function which is Boolean variable that is equal to 1 or 0.

Register Transfer



(a) Block diagram



(b) Timing diagram

Register Transfer

- The n outputs of register R1 are connected to the n inputs of register R2. The letter n will be used to indicate any number of bits for the register.
- It is assumed that all transfers occur during a clock edge transition. Even though the control condition such as P becomes active just after time t, the actual transfer does not occur until the register is triggered by the next positive transition of the clock at time t + 1.
- A comma is used to separate two or more operations that are executed at the same time.The statement

$$T: R2 \leftarrow R1, R1 \leftarrow R2$$

denotes an operation thatexchanges the contents of two registers during one common clock pulse provided that T = 1.

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	*MAR*, *R2*
Parentheses ( )	Denotes a part of a register	*R2*(0–7), *R2*(*L*)
Arrow ←	Denotes transfer of information	*R2* ← *R1*
Comma ,	Separates two microoperations	*R2* ← *R1*, *R1* ← *R2*

- For example, RTL of fetch cycle can be written as: T1: MAR←PC
  T2: MBR←[MAR]
  T3: IR←MBR
  T4: unspecified; PC←PC+1

- The notation (T1, T2, T3, T4) represents successive time units.
- All three units are of equal duration. A time unit is defined by regularly spaced clock pulses. The operations performed within this single unit of time are called microoperations.
- A single time unit can contain one or more microoperations. Since each microoperation can specifies the transfer of data into or out of a register, such type is called **RTL.**

Bus and Memory Transfer

Bus

- A typical digital computer has many registers, and paths must be provided to transfer information from one register to another.
- The number of wires will be excessive if separate lines are used between each register and all other registers in the system.
- A more efficient scheme for transferring information between registers in a multiple register configuration is a common bus system.
- A bus structure consists of a set of common lines, one for each bit of a register, through which binary information is transferred one at a time.
- Control signals determine which register is selected by the bus during each particular register transfer.

Bus and Memory Transfer

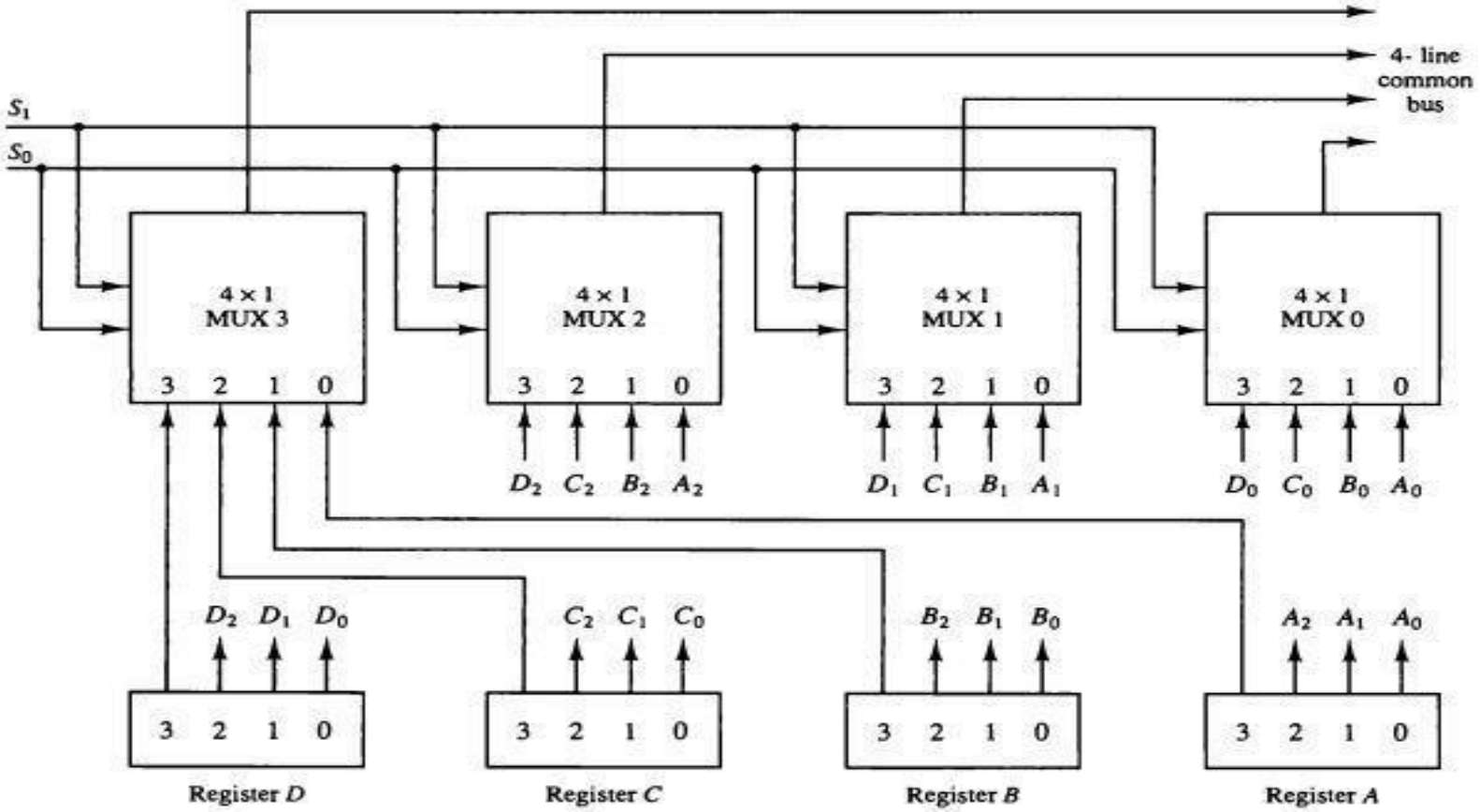


Fig: Bus system for four registers.

Bus and Memory Transfer

- Here, the bus contain 4 X 1 MUX and each have four data inputs, 0 through 3, and two selection inputs S_1 and S_0 .
- for example, output 1 of register A is connected to input 0 of MUX 1 because this input is labeled A_1 .
- The two selection lines S_1 and S_0 are connected to the selection inputs of all four multiplexers.
- The selection lines choose the four bits of one register and transfer them into the four-line common bus.

Bus and Memory Transfer

- When $S_1 S_0 = 00$, the 0 data inputs of all MUX are selected and applied to the output that form the bus.
- This causes the bus lines to receive the content of register A since the output of this register are connected to the 0 data inputs of the MUX.
- Similarly, register B is selected
- if $S_1S_0 = 01$, and so on.

S_1	S_0	Register selected
0	0	A
0	1	B
1	0	C
1	1	D

Fig: Function table for Bus

Bus and Memory Transfer

- In general, a bus system will multiplex **k** registers of **n** bits each to produce an **n** line common bus.
- The number of multiplexers needed to construct the bus is equal to **n**, the number of bits in each register.
- The size of each multiplexer must be **k X 1** since it multiplexes k datalines.
- For example, a common bus for eight registers of **16** bits each requires **16** multiplexers, one for each line in the bus.
- Each multiplexer must have **eight** data input lines and **three** selection lines to multiplex one significant bit in the eight registers.
- The transfer of information from a bus into one of many destination registers can be accomplished by connecting the bus lines to the inputs of all destination registers and activating the load control of the particular destination register selected.

Memory Transfer

- The transfer of information from a memory word to the outside environment is called a **read operation**.
- The transfer of new information to be stored into the memory is called a **write operation**.
- A memory word will be symbolized by the letter **M**. The particular memory word among the many available is selected by the memory address during the transfer.
- Consider a memory unit that receives the address from a register, called the **address register**, symbolized by **AR**. The data are transferred to another register, called the **data register**, symbolized by **DR**. The read operation can be stated as follows:

  Read: DR ← M[AR]

  This causes a transfer of information into DR from the memory word M selected by the address in AR.

Memory Transfer

- The write operation transfer the content of a data register to a memory word M selected by the address.
- Assume that the input data are in register R1 and the address is in AR.
- The write operation can be started symbolically as follows:

Write : M[AR]← R1

- The cause a transfer of information from R1 into the memory word M selected by the address in AR.

Microoperations

- The operations on the data in registers are called microoperations.
- The result of the operation may replace the previous binary information of the resister or may be transferred to another resister.
- Register Transfer Language (RTL) can be used to describe the sequence of micro-operations.
- The microoperations most often encountered in digital computers are classified into 4 categories:
  i)Register transfer microoperations
  ii)Arithmetic microoperations
  iii)Logic microoperations
  iv)Shift microoperations

Register transfer microoperations

- Registers are designated by capital letters, sometimes followed by numbers (e.g., A, R13, IR). Often the names indicate function:

MAR	memory address register
PC	program counter
IR	instruction register

- Information described in transfer from one register to another is symbolic form by replacement operator.
- The statement "R2 ←R1" denotes a transfer of the content of the R1 into resister R2.

Control Function

- Often actions need to only occur if a certain condition is true. In digital systems, this is often done via a control signal, called a control function.

Example: P: R2 ← R1 i.e. if (P = 1) then (R2← R1)

Which means "if P = 1, then load the contents of register R1 into register R2".

- If two or more operations are to occur simultaneously, they are separated with commas.

Example: P: R3 ← R5, MAR ← IR

Arithmetic microoperations

The basic arithmetic microoperations are

- Addition
- Subtraction
- Increment
- Decrement

The additional arithmetic microoperations are

- Add with carry
- Subtract with borrow
- Transfer/Load

Arithmetic microoperations

Summary of typical arithmetic microoperations

Symbolic Designation	Description
R3 ← R1 + R2	Contents of R1 plus R2 transferred to R3
R3 ← R1 + R2' + 1	Contents of R1 minus R2 transferred to R3
R1 ← R1 + 1	Increment the contents of R1 by one
R1 ← R1 - 1	Decrement the contents of R1 by one
R3 ← R1 + R2 + 1	Add with carry
R3 ← R1 + R2'	Subtract with borrow
R1 ← R1' +1	2's complement the contents of R1 (negate)

Logic microoperations

- Logic microoperations are bit-wise operations, i.e., they work on the individual bits of data.
- Useful for bit manipulations on binary data and for making logical decisions based on the bit value.
- There are, in principle, 16 different logic functions that can be defined over two binary input variables.
- However, most systems only implement four of these

– AND (/\), OR (\/), XOR ($\oplus$), Complement/NOT

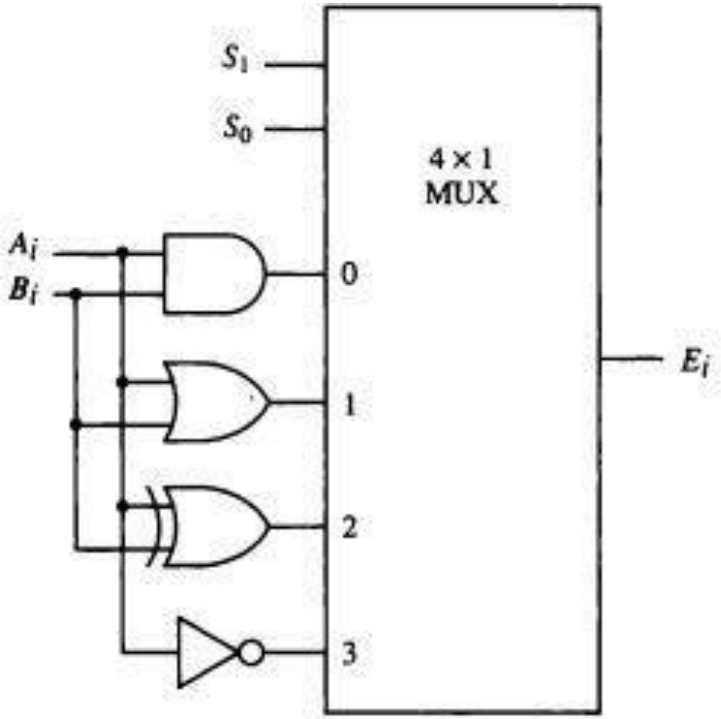
Logic microoperations

- The others can be created from combination of these four functions

Microoperation	Name
F ← R1 /\ R2	AND
F ← R1 \/ R2	OR
F ← R1 ⊕ R2	XOR
F ← R1’	Complement (NOT)

Logic microoperations

Hardware Implementation



(a) Logic diagram

S_1	S_0	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \overline{A}$	Complement

(b) Function table

Shift microoperations

- The operation that changes the adjacent bit position of the binary values stored in the register is known as shift microoperation.
- They are used for serial transfer of data.
- The shift microoperations are classified into 3 types:

  i) Logical shift

  ii)Circular shift

  iii)Arithmetic shift

Shift microoperations

Logical shift: A logical shift transfer 0 through the serial input. It can be defined in RTL by:

R←shl R	shift-left register R
R←shr R	shift-right register R

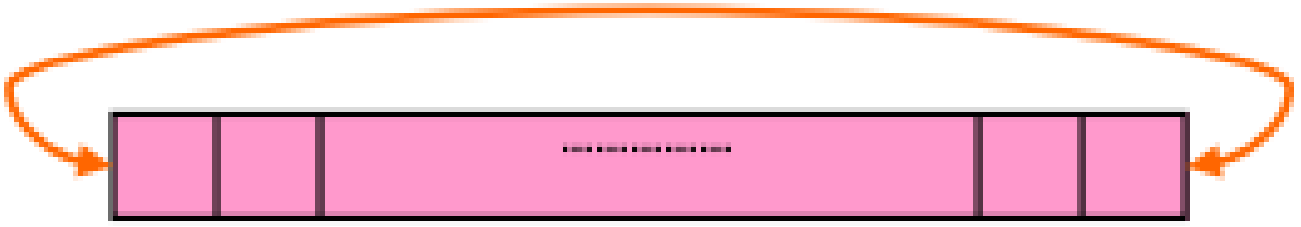


Shift microoperations

Circular shift: A circular shift rotates the bit from one end of the register to another end of the register. It can be defined in RTL by:

R←cil R circular shift-left register R

R←cir R circular shift-right register R

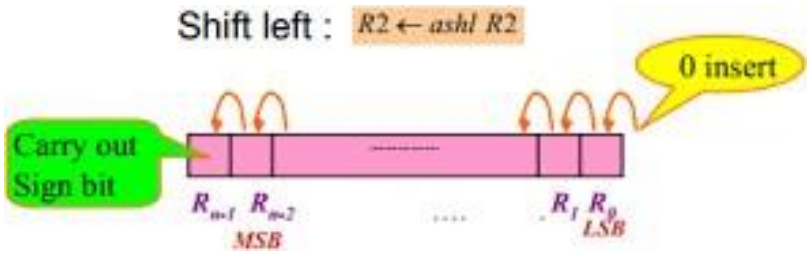


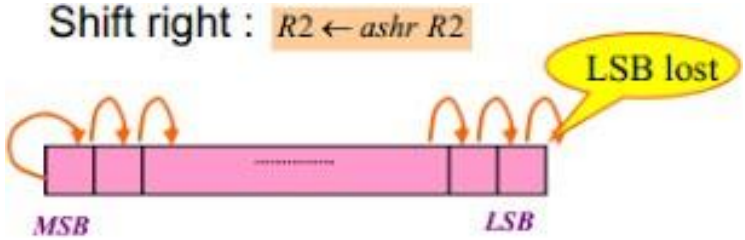
Shift microoperations

Arithmetic shift: It shifts signed-binary number left or right. For shift left the content of the register is multiplied by 2 whereas For shift right the content of the register is divided by 2. The arithmetic shift must leave the sign bit unchanged. It can be defined in RTL by:

R←ashl R arithmetic shift-left register R

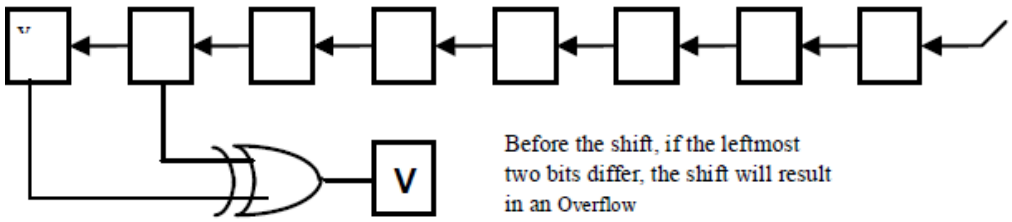
R←ashr R arithmetic shift-right register R





Overflow case during arithmetic shift left:

- If a bit in R_{n-1} changes in value after the shift, sign reversal occurs in the result. This happens if the multiplication by 2 causes an overflow.
- Thus, left arithmetic shift operation must be checked for the overflow: an overflow occurs after an arithmetic shift-left if before shift $R_{n-1} \neq R_{n-2}$.



Before the shift, if the leftmost two bits differ, the shift will result in an Overflow

An overflow flip-flop V can be used to detect an arithmetic shift-left overflow.

$$V = R_{n-1} \oplus R_{n-2}$$

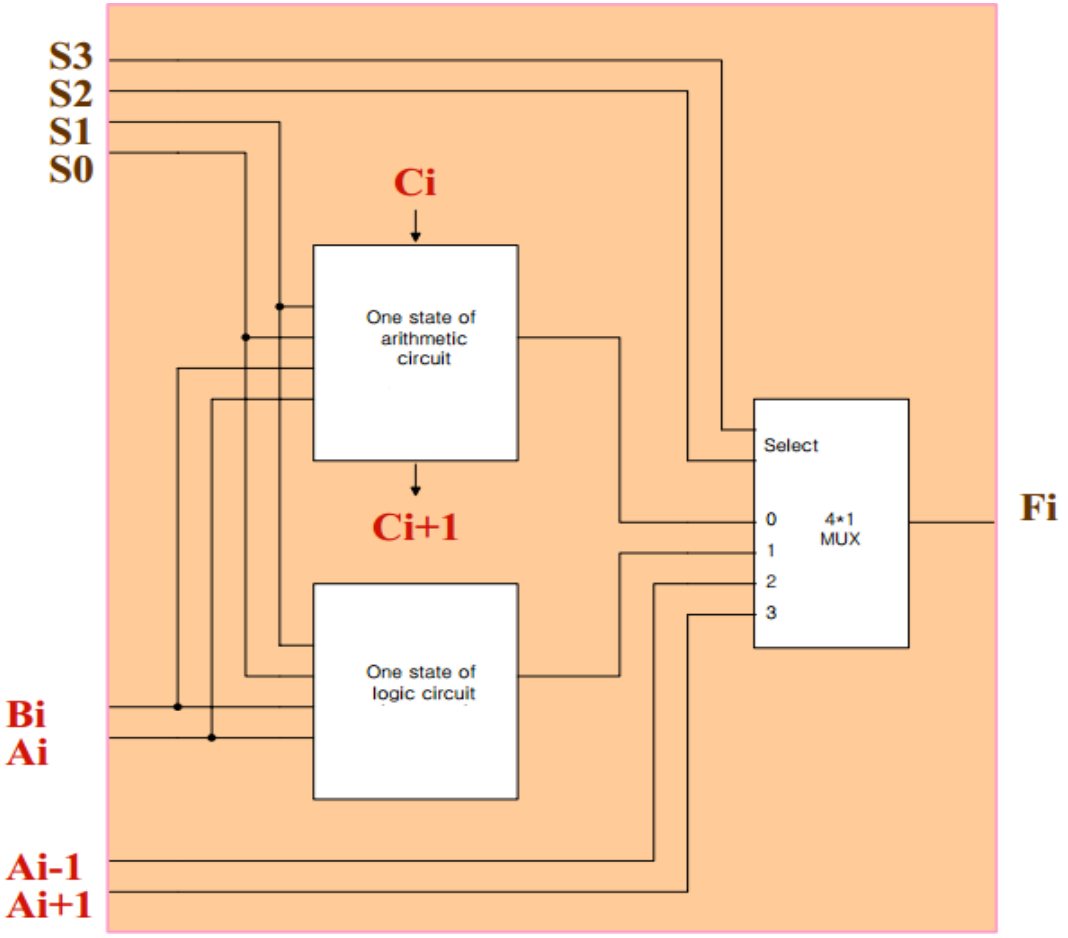
If V = 0, there is no overflow but if V = 1, overflow is detected.

Arithmetic Logic Shift Unit

- Arithmetic logic shift unit is a digital circuit that performs arithmetic calculations, logical manipulation and shift operation.
- It is often abbreviated as ALU.
- The block diagram of ALU includes one stage of arithmetic circuit, one stage of logic circuit and one 4*1 multiplexer.
- The subscript i designates a typical stage.

Arithmetic Logic Shift Unit

Figure shows the one state of arithmetic logic shift unit.



Arithmetic Logic Shift Unit

- Inputs A_i and B_i are applied to both the arithmetic and logic units. A particular microoperation is selected with inputs S_1 and S_0.
- A 4 x 1 MUX selects the final output.
- The two inputs of the MUX are received from the output of the arithmetic circuit and logic circuit.
- The other two is A_{i-1} for the shift-right operation and A_{i+1} for the shift left operation.
- The circuit is repeated n times for n-bit ALU.
- The output carry C_{i+1} is connected to the input carry C_{in}.
- In every stage the circuit specifies 8 arithmetic operations, 4 logical operations and 2 shift operations, where each operation is selected by the five variables S_3, S_2, S_1, S_0 and C_{in}.